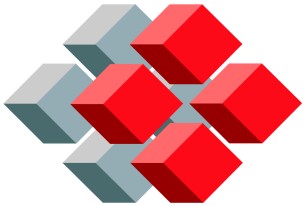
ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

CSCS
Swiss National Supercomputing Centre

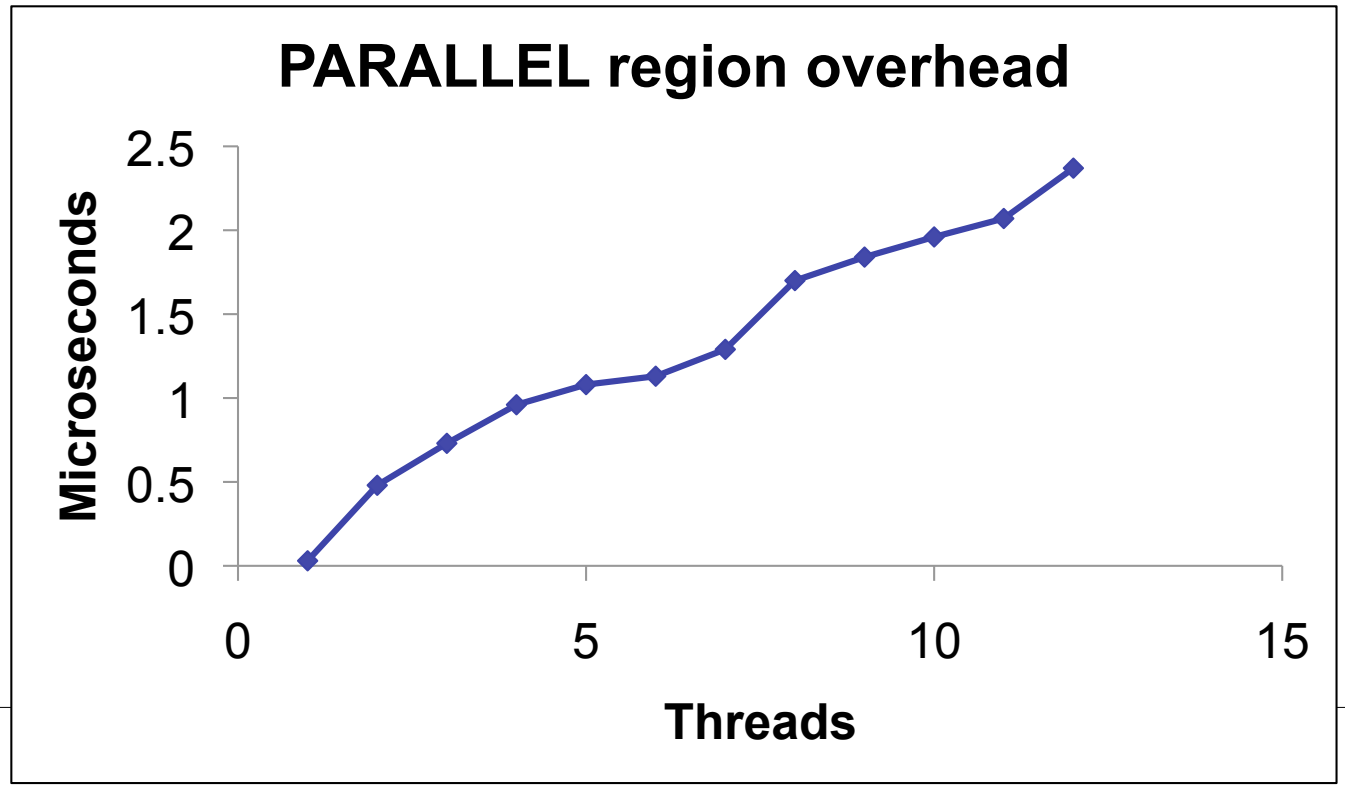


OpenMP Optimization

National Supercomputing Service
Swiss National Supercomputing Center

Parallel region overhead

- Creating and destroying parallel regions takes time.





Avoid too many parallel regions

- Overhead of creating threads adds up
- Can take a long time to insert hundreds of directives
- Software engineering issues
  - Adding new code to a parallel region means making sure new private variables are accounted for.
- Try using one large parallel region with do loops inside or hoist one loop index out of a subroutine and parallelize that

Parallel regions example

Instead of this….

```
SUBROUTINE foo()
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
!$OMP PARALLEL DO…
END SUBROUTINE foo
```

Do this…..

```
SUBROUTINE foo()
!$OMP PARALLEL
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP DO…
!$OMP END PARALLEL
END SUBROUTINE foo
```

Or this…

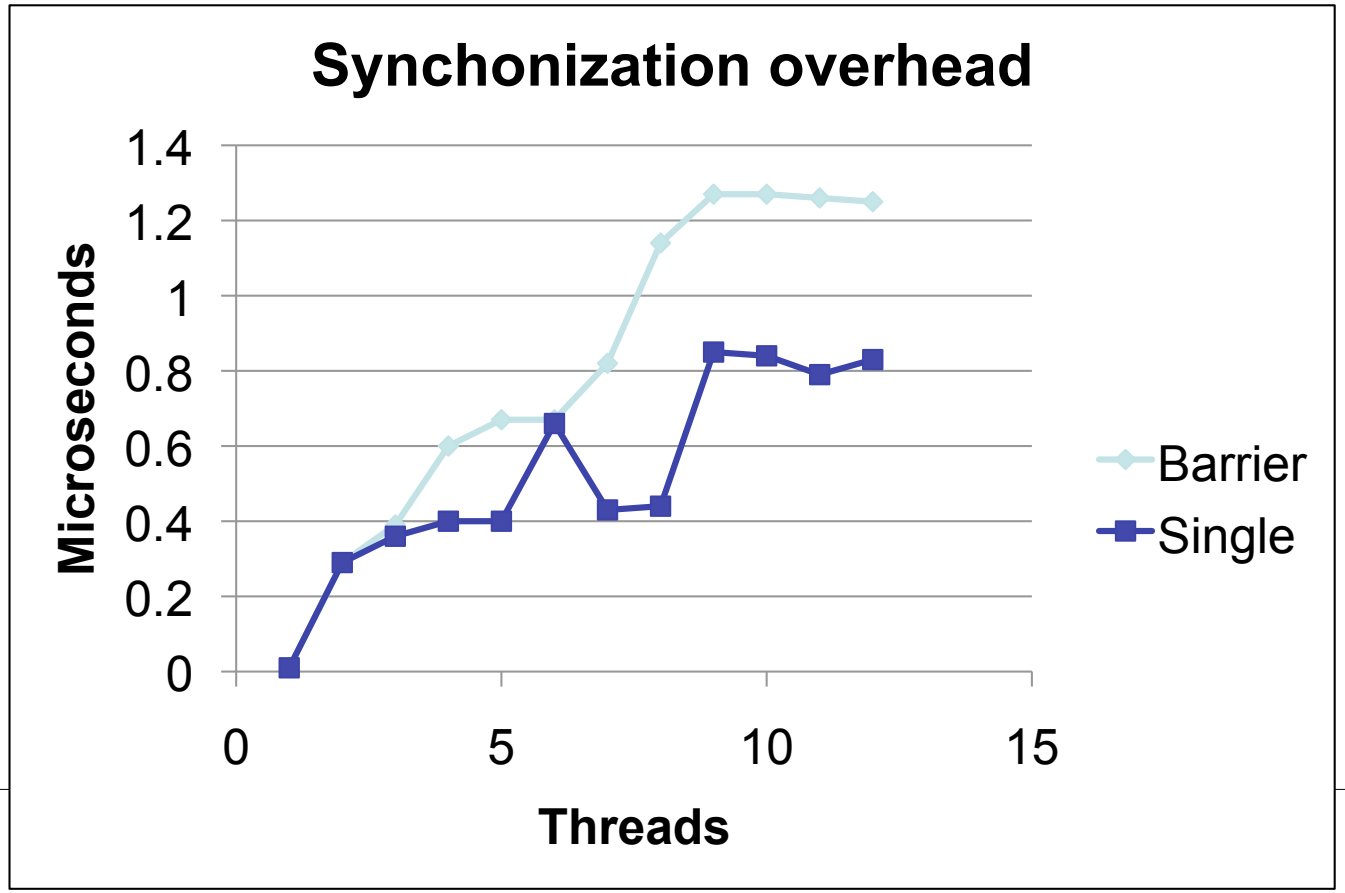
```
!$OMP PARALLEL DO
DO I = 1, N
   CALL foo(i)
END DO
!$OMP END PARALLEL DO


SUBROUTINE foo(i)
   …many do loops…
END SUBROUTINE foo
```

Hoisting a loop out of the subroutine….

Synchronization overhead

- Synchronization barriers cost time!





Minimize sync points!

- Eliminate
- Use `master` instead of `single` since master does not have an implicit barrier.
- Use thread private variables to avoid `critical/atomic` sections
  - e.g. promote scalars to vectors indexed by thread number.
- Use `NOWAIT` directive if possible.
  - `!$OMP END PARALLEL DO NOWAIT`



Load balancing

- Examine work load in loops and determine if `dynamic` or `guided` scheduling would be a better choice.
- In nested loops, if outer loop counts are small, consider collapsing loops with `collapse` directive.
- If your work patterns are irregular (e.g. server-worker model), consider nested or tasked parallelism.



Parallelizing non-loop sections

- By Amdahl's law, anything you don't parallelize will limit your performance.
- It may be that after threading your do-loops, your run-time profile is dominated by non-parallelized non-loop sections.
- You might be able to parallelize these by using OpenMP sections or tasks.



Non-loop example

```
/* do loop section */
#pragma omp parallel sections
#pragma omp section
{
    thread_A_func_1();
    thread_A_func_2();
}

#pragma omp section
{
    thread_B_func_1();
    thread_B_func_2();
}

} /* implicit barrier */
```



Memory performance

- Most often, the scalability of shared memory programs is limited by the movement of data.
  - For MPI-only programs, where memory is compartmentalized, memory access is less of an explicit problem, but not unimportant.
- On shared-memory multicore chips, the latency and bandwidth of memory access depends on their locality.
- Achieving good speedup means Locality is King.



Locality

- Initial data distribution determines on which CPU data is placed
  - *first touch* memory policy (see next)
- Work distribution (i.e. scheduling)
  - Chunk size
- “Cache friendliness” determines how often main memory is accessed (see next)



First touch policy (page locality)

- Under Linux, memory is managed via a *first touch* policy.
  - Memory allocation functions (e.g. `malloc`,`ALLOCATE`) don't actually allocate your memory. This is done when a processor first tries to access a memory reference.
  - Problem: Memory will be placed on the core that 'touches' it first.
- For good spatial locality, best to have the memory a processor needs on the same CPU.
  - Initialize your memory as soon as you allocate it.

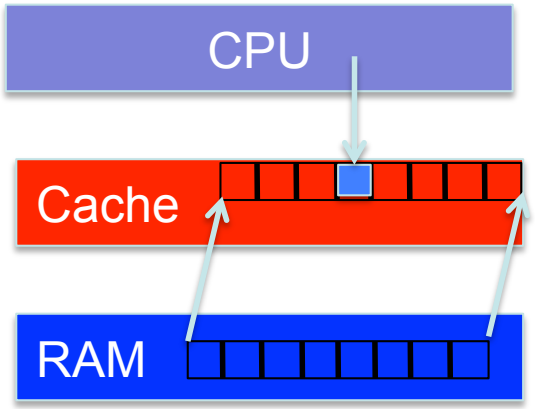


Work scheduling

- Changing the type of loop scheduling, or changing the chunk size of your current schedule, may make your algorithm more cache friendly by improving spatial and/or temporal locality.
  - Are your chunk sizes ‘cache size aware’? Does it matter?

Cache….what is it good for?

- On CPUs, *cache* is smaller/faster memory buffer which stores copies of data in the larger/slower main memory.
- When the CPU needs to read or write data, it first checks to see if it is in the cache instead of going to main memory.
- If it isn't in cache, accessing a memory reference (e.g. A(i), an array element) loads in not only that piece of memory but an entire section of memory called a *cache line* (64 bytes for Istanbul chips).
- Loading a cache line improves performance because it is likely that your code will use data adjacent to that (e.g. in loops: … A(i-2) A(i-1) A(i) A(i+1) A(i+2))





Cache friendliness

- Locality of references
  - Temporal locality: data is likely to be reused soon. Reuse same cache line. (might use *cache blocking*)
  - Spatial locality: adjacent data is likely to be needed soon. Load adjacent cache lines.
- Low cache contention
  - Avoid sharing of cache lines among different threads (may need to increase array sizes or ranks) (see False Sharing)



Spatial locality

- The best kind of spatial locality is where your next data reference is adjacent to you in memory, e.g. stride-1 array references.
- Try to avoid striding across cache lines (e.g. matrix-matrix multiplies). If you have to try to
  - Refactor your algorithm for stride-1 arrays
  - Refactor your algorithm to use *loop blocking* so that you can improve data reuse (temporal locality)
    - E.g. decomposing a large matrix into many smaller blocks and using OpenMP on the number of blocks rather than on the array indices themselves.



Loop blocking

Unblocked

```
DO k = 1, N3
     DO j = 1, N2
          DO i = 1, N1
               ! Update f using some
               ! kind of stencil
               f(i,j,k) = …
          END DO
     END DO
END DO
```

Blocked in two dimensions

```
DO KBLOCK = 1, N3, BS3
DO JBLOCK = 1, N2, BS2
DO k = KBLOCK, MIN(KBLOCK+BS3-1,N3)
     DO j = JBLOCK,MIN(JBLOCK+BS2-1,N2)
          DO i = 1,N1
               f(i,j,k) = …
          END DO
     END DO
END DO
END DO
END DO
```

- Stride-1 innermost loop = good spatial locality.
- Loop over blocks on outermost loop = good candidate for OpenMP directives

- Independent blocks with smaller size = better data reuse (temporal locality)
- Experiment to tune block size to cache size.
- Compiler may do this for you.



Common blocking problems (J.Larkin,Cray)

- Block size too small
  - too much loop overhead
- Block size too large
  - Data falling out of cache
- Blocking the wrong set of loops
- Compiler is already doing it
- Computational intensity is already large making blocking unimportant



False Sharing (cache contention)

- What is it?
- How does it affect performance?
- What does this have to do with OpenMP?
- How to avoid it?

Example 1

```
int val1, val2;

Void func1() {
    val1 = 0;
    for(i=0; i<N; i++){
        val1 += …;
    }
}
Void func2() {
    val2 = 0;
    for(i=0; i<N; i++ ){
        val2 += …;
    }
}
```

Because val1 and val2 are adjacent to each other in their declaration, they will likely be allocated next to each other in memory in the same cache line.

Val1 locks cache. Val2 then shares it. func1 updates val1, invalidating func2's cache.

Func2 updates val2, but it has a coherence miss so it invalidates val1's cache, forcing func1 to write back to memory

Func1 reads val1 again but it's cache is invalidate by func2 forcing func2 to do a write back to memory.



How to avoid it?

- Avoid sharing cache lines.
- Work with thread private data.
  - May need to create private copies of data or change array ranks.
- Align shared data with cache boundaries.
  - Increase problem size or change array ranks
- Change scheduling chunk size to give each thread more work.
- Use optimization of compiler to eliminate loads and stores.



Task/thread migration (affinity)

- The compute node OS can migrate tasks and threads from one core to another within a node.
- In some cases, because of where your allocated memory may be placed (first touch), moving tasks and threads may cause a decrease in performance .



CPU affinity

- Options for the aprun command enable the user to bind a task or a thread to a particular CPU or subset of CPUs on a node.
  - -cc cpu: binds tasks to CPUs with the assigned NUMA node.
  - -ss : a task can only allocate memory local to its NUMA node.
  - If tasks create threads, the threads are constrained to the same NUMA-node CPUs as the tasks.
    - If num_threads > num_cpus per NUMA node CPU then additional threads are bound to the next NUMA node.